Resource-to-Need Alignment Matters: How Equity-Targeted Resources Shape Student Outcomes in CPS

Andi Fang Minyang Xu

2026-03-03

Group Information

- **Group Members:** Andi Fang, Minyang Xu
- **Lecture Section:** Tue/Thu 9:30am - 10:50am, Section 3
- **GitHub Usernames:** andifang-123(Andi Fang), clorisaaaaamelody(Minyang Xu)
- **Repository:** andifang-123/adf-myx-gp

1. Research Motivation & Question

Our project investigates how school resources and structural inequalities relate to student performance in Chicago Public Schools (CPS). Motivated by personal observations—one teammate worked with CPS through an internship, and the other observed strong on-campus facilities at a nearby school—we ask a single guiding question: **Are school resources aligned with student need, and how does this alignment relate to standardized academic outcomes (ELA and Math)?**

2. Approach & Methodology

To move beyond simple per-student funding metrics, we developed two composite indices.

The Resource Index (RI) – Measuring Capacity

RI combines physical (**Hard**) and institutional (**Soft**) resources, weighted equally.

- **Hard Resource Index:** We used **Space Utilization (SU)** as a proxy for physical capacity because facility "stock value" data were largely missing. We defined an efficiency center at (c = 0.85) with an overcrowding penalty factor gamma = 1.7, which is more harmful to learning.

  Distance to efficient use is defined as:

$$d(SU) = \begin{cases} c - SU, & SU < 0.7 \\ |SU - c|, & 0.7 \leq SU \leq 1.0 \\ \gamma(SU - c), & SU > 1.0 \end{cases}$$

The hard-resource score is:

$$Hard = -z(d(SU)).$$

- **Soft Resource Index:** We adopted the **5Essentials (5E)** framework—a research-based annual school improvement survey that measures five organizational conditions linked to student outcomes. It captures school climate and capacity across: Ambitious Instruction (AI), Effective Leaders (EL), Collaborative Teachers (CT), Involved Families (IF), and Supportive Environment (SE). Each domain is standardized via z-scores and averaged:

$$Soft = \frac{1}{5}\left(z_{AI} + z_{EL} + z_{CT} + z_{IF} + z_{SE}\right).$$

- **Final Resource Index (RI):**

$$RI = 0.5\,Hard + 0.5\,Soft.$$

Interpretation: Higher **Resource Index (RI)** indicates stronger overall school capacity—both physical capacity/availability and stronger organizational (5E) conditions.

- **The Opportunity Constraint Index (OI) – Measuring Need**

OI quantifies student need by averaging z-scores across four domains: Economic Disadvantage %, English Learners %, Disabilities %, and Race Concentration (measured via a Simpson-based concentration index):

$$Simpson = \sum_r p_r^2.$$

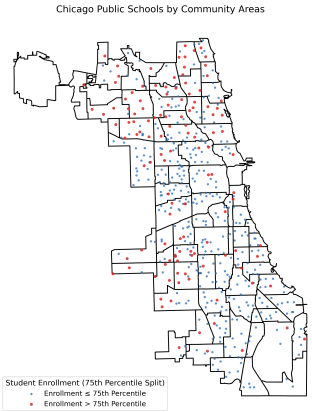
Higher **OI** indicates greater opportunity constraints (and thus greater resource need to achieve comparable outcomes).

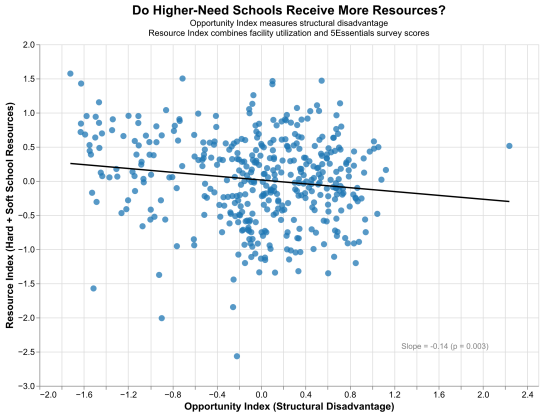
- **Weaknesses**
  - **Resource measurement limits:** Hard resources are proxied by space utilization; we lack granular data on direct spending or staffing counts.
  - **Index construction assumptions:** OI assumes equal weights across components, though economic disadvantage might plausibly deserve a higher weight.
  - **Interpretation scope:** Our results are descriptive and do not establish causal effects of resources on outcomes.

3. Static Visualizations

Our static plots provide the baseline for our structural analysis:

- **Spatial Distribution of Public Schools:** This map shows that schools are unevenly distributed. Schools with higher enrollment (above the 0.75 quantile) are visibly clustered in the North and West community areas compared to southern and central areas. *(left figure)*
- **Do Higher-Need Schools Receive More Resources? (RI vs OI):** The RI–OI scatter plot provides a baseline diagnostic of resource–need alignment by comparing the **Resource Index (RI)** against the **Opportunity Constraint Index (OI)** across schools. *(right figure)*





4. Streamlit Applications

We developed two interactive apps to explore these dynamics in depth:

- **Subgroup Distribution App:** (https://adf-myx-gp-structural-disadvantage.streamlit.app/)

This tool allows users to toggle between ELA and Math results to see if performance differs systematically across disadvantage groups. It visualizes that structural barriers are strongly reflected in proficiency distributions.

- **Spatial & Scatter Explorer** (https://adf-myx-gp-sptial-scatter.streamlit.app/):
  - **Spatial View:** Maps community-average **Resource Index (RI)** and **Opportunity Index (OI)** scores, listing the top and bottom 10 communities. This enables cross-comparisons to identify mismatch areas (e.g., **low RI but high OI**, or **high RI but low OI**) for further review.
  - **Regression View:** Links academic outcomes (**ELA/Math proficiency**) to RI and OI. Across the four scatter plots, the fitted lines summarize the substantive pattern: **higher RI/lower OI is generally associated with higher proficiency**, highlighting how resources and constraints relate to student outcomes.

5. Substantive Results & Conclusion

Our primary finding is **resource–need misalignment**: the relationship between **Resource Index (RI)** and **Opportunity Index (OI)** is flat or negative, whereas a need-aligned system would be **non-negative on average** (higher constraints should not systematically coincide with lower resources). We do not claim a single "correct" numeric slope, but under our framework the expected **direction** is non-negative (potentially nonlinear), so a flat/negative pattern is substantively meaningful as a diagnostic of potential misallocation.

Policy Recommendation: Use **OI** and **RI** jointly to target and re-balance support — prioritize **high-need, low-resource** communities for additional investment, review **low-need, high-resource** areas for potential re-targeting, and move toward a need-aligned allocation rule where OI helps guide investments and track whether gaps narrow over time.

Appendix: Related Code

1. Preprocessing & Index Construction

Introduction

This project constructs a unified dataset of Chicago Public Schools (CPS) by integrating administrative records, academic performance measures, demographic characteristics, and survey-based institutional indicators.

We develop three composite indices:

- **Resource Index (RI)**: combining physical and institutional capacity
- **Simpson Diversity Index**: measuring racial concentration
- **Opportunity Index (OI)**: capturing cumulative student disadvantage

These measures allow us to examine patterns of resource allocation and student disadvantage across schools. The resulting dataset is saved for downstream analysis.

Environment Setup

```
import pandas as pd
from pathlib import Path
import numpy as np
from scipy.stats import zscore
import os

project_root = Path(os.environ.get("QUARTO_PROJECT_DIR", Path.cwd())).resolve()
raw_dir = project_root / "data" / "raw-data"
derived_dir = project_root / "data" / "derived-data"
```

All paths are defined relative to the project root to ensure reproducibility.

Helper Functions

```
def clean_id(series):
    return (
        series.astype(str)
        .str.replace(",", "", regex=False)
        .str.replace(".0", "", regex=False)
        .str.strip()
    )

def load_excel_merge(path, sheet, cols, id_col):
    df = pd.read_excel(path, sheet_name=sheet, usecols=cols)
    df = df.rename(columns={id_col: "School_ID"})
    df["School_ID"] = clean_id(df["School_ID"])
    return df
```

These functions standardize school identifiers across datasets before merging.

Base Dataset Construction

```
cps = pd.read_csv(raw_dir / "cps_info.csv", sep=";")

cps["School_ID"] = clean_id(cps["School_ID"])
cps["Zip"] = clean_id(cps["Zip"])

cps = cps[
    [
        "School_ID",
        "Short_Name",
        "Long_Name",
        "Primary_Category",
        "Zip",
        "School_Latitude",
        "School_Longitude",
    ]
]

cps.head()
```

This base dataset provides structural school-level identifiers and geographic information.

Merging Institutional and Demographic Data

5Essentials Survey (Institutional Quality)

```
ess = load_excel_merge(
    raw_dir / "5essentials.xlsx",
    "Data",
    [
        "schoolID",
        "Ambitious Instruction",
        "Effective Leaders",
        "Collaborative Teachers",
        "Involved Families",
        "Supportive Environment",
    ],
    "schoolID",
)

ess["Involved Families"] = pd.to_numeric(ess["Involved Families"], errors="coerce")
cps = cps.merge(ess, on="School_ID", how="left")
```

These variables measure soft institutional capacity and school climate.

Space Utilization (Hard Resources)

```
space = load_excel_merge(
    raw_dir / "spaceuse.xlsx",
    "Data",
    [
        "School ID",
        "Total Classrooms (CRs)",
        "SY2026 Adjusted SU 2",
    ],
    "School ID",
)

cps = cps.merge(space, on="School_ID", how="left")
```

Space utilization captures physical capacity pressure within schools.

Student Demographics

```
lif = load_excel_merge(
    raw_dir / "demo_lepiepfrm.xlsx",
    "Schools",
    [
        "School ID",
        "State English Learners N",
        "State English Learners %",
        "Students with Disabilities N",
        "Students with Disabilities %",
        "Economically Disadvantaged N",
        "Economically Disadvantaged %",
    ],
    "School ID",
)

cps = cps.merge(lif, on="School_ID", how="left")
```

These measures capture dimensions of student disadvantage.

Racial Composition

```
race = load_excel_merge(
    raw_dir / "demo_racialethnic.xlsx",
    "Schools",
    [
        "School ID",
        "Total",
        "White N",
        "Black/African American N",
        "Latinx N",
```

```
        "Not Available N",
    ],
    "School ID",
)

cps = cps.merge(race, on="School_ID", how="left")

cps["Others N"] = (
    cps["Total"]
    - cps["White N"]
    - cps["Black/African American N"]
    - cps["Latinx N"]
    - cps["Not Available N"]
)

denom = (cps["Total"] - cps["Not Available N"]).replace(0, np.nan)

cps["White %"] = cps["White N"] / denom
cps["Black %"] = cps["Black/African American N"] / denom
cps["Latinx %"] = cps["Latinx N"] / denom
cps["Others %"] = cps["Others N"] / denom

cps = cps.drop(
    columns=[
        "White N",
        "Black/African American N",
        "Latinx N",
        "Others N",
        "Not Available N",
    ]
)
```

We convert raw counts into proportional racial shares.

IAR Academic Performance

```
iar = pd.read_csv(raw_dir / "iar_proficiency.csv")

iar_filtered = iar.loc[
    iar["Test Name"].isin(["IAR ELA", "IAR MATH"]) &
    (iar["Student Group"] == "All Students"),
    ["School Code", "Test Name", "%At or Above Proficient"]
].copy()
iar_filtered["%At or Above Proficient"] = pd.to_numeric(
    iar_filtered["%At or Above Proficient"], errors="coerce"
)

iar_wide = (
    iar_filtered
    .pivot(index="School Code", columns="Test Name", values="%At or Above Proficient")
```

```
    .reset_index()
    .rename(columns={
        "School Code": "School_ID",
        "IAR ELA": "IAR ELA %",
        "IAR MATH": "IAR MATH %"
    })
)

iar_wide["School_ID"] = clean_id(iar_wide["School_ID"])
cps = cps.merge(iar_wide, on="School_ID", how="left")

iar["School_ID"] = clean_id(iar["School Code"])
cps = cps[cps["School_ID"].isin(iar["School_ID"])]
```

We incorporate standardized test performance using IAR proficiency rates in English Language Arts (ELA) and Mathematics. Only "All Students" results are retained to ensure comparability across schools.

To ensure consistency in outcome measurement, we restrict the sample to schools appearing in the IAR dataset. This effectively limits the analysis to elementary schools with standardized test performance data.

Index Construction

Resource Index (RI)

```
c = 0.85        # efficient center
gamma = 1.7     # crowding penalty

su = cps["SY2026 Adjusted SU 2"]

d = np.where(
    su < 0.7,
    c - su,
    np.where(
        su <= 1.0,
        np.abs(su - c),
        gamma * (su - c),
    ),
)

cps["SU_distance"] = d
cps["Hard Resource Index"] = -zscore(cps["SU_distance"], nan_policy="omit")
cps = cps.drop(columns=["SU_distance"])

fiveE = [
    "Ambitious Instruction",
    "Effective Leaders",
    "Collaborative Teachers",
    "Involved Families",
```

```
    "Supportive Environment",
]

for col in fiveE:
    cps[f"{col}_z"] = zscore(cps[col], nan_policy="omit")

z_cols = [f"{c}_z" for c in fiveE]
cps["Soft Resource Index"] = cps[z_cols].mean(axis=1)
cps = cps.drop(columns=z_cols)

cps["RI"] = 0.5 * cps["Hard Resource Index"] + 0.5 * cps["Soft Resource Index"]
```

The Resource Index combines physical and institutional capacity.

Simpson Diversity Index

```
race_pct_cols = ["White %", "Black %", "Latinx %", "Others %"]
for c in race_pct_cols:
    cps[c] = pd.to_numeric(cps[c], errors="coerce")

cps["Simpson Diversity Index"] = (cps[race_pct_cols] ** 2).sum(axis=1)
```

Higher values indicate greater racial concentration.

Opportunity Index (OI)

```
ed_col = "Economically Disadvantaged %"
ell_col = "State English Learners %"
dis_col = "Students with Disabilities %"
race_index_col = "Simpson Diversity Index"

required_cols = [ed_col, ell_col, dis_col, race_index_col]
for c in required_cols:
    cps[c] = pd.to_numeric(cps[c], errors="coerce")

def zscore_series(s):
    mu = s.mean(skipna=True)
    sigma = s.std(skipna=True, ddof=0)
    if pd.isna(sigma) or sigma == 0:
        return pd.Series(np.nan, index=s.index)
    return (s - mu) / sigma

cps["Z_ED"] = zscore_series(cps[ed_col])
cps["Z_ELL"] = zscore_series(cps[ell_col])
cps["Z_DIS"] = zscore_series(cps[dis_col])
cps["Z_RACE"] = zscore_series(cps[race_index_col])

cps["OI"] = (
```

```
    cps["Z_ED"] + cps["Z_ELL"] + cps["Z_DIS"] + cps["Z_RACE"]
) / 4

cps = cps.drop(columns=["Z_ED", "Z_ELL", "Z_DIS", "Z_RACE"])
```

The Opportunity Index summarizes cumulative student disadvantage exposure.

Save Final Dataset

```
derived_dir.mkdir(parents=True, exist_ok=True)
output_path = derived_dir / "cps_df.csv"
cps.to_csv(output_path, index=False)

print(f"Final CPS dataset saved: {output_path}")
print(f"Rows: {len(cps)}, Columns: {len(cps.columns)}")
cps.head(5)
```

2. CPS Spatial Distribution Plot

Introduction

This document visualizes Chicago Public Schools (CPS) on top of Chicago community boundaries. Schools are split into two groups using an enrollment threshold (the 0.75 quantile of total enrollment), and plotted with different marker colors.

- **Blue points**: schools at/below the 0.75 quantile threshold
- **Red points**: schools above the 0.75 quantile threshold

```
project_root = Path(os.environ.get("QUARTO_PROJECT_DIR", Path.cwd())).resolve()

raw_community = project_root / "data" / "raw-data" / "community_boundaries.csv"
raw_schools   = project_root / "data" / "derived-data" / "cps_df.csv"
```

Load Data

```
comm_df = pd.read_csv(raw_community, sep=";")
schools_df = pd.read_csv(raw_schools, sep=None, engine="python")
```

Convert to GeoDataFrames

```
comm_gdf = gpd.GeoDataFrame(
    comm_df,
    geometry=comm_df["the_geom"].apply(wkt.loads),
    crs="EPSG:4326",
)

schools_gdf = gpd.GeoDataFrame(
    schools_df,
    geometry=gpd.points_from_xy(schools_df["School_Longitude"], schools_df["School_Latitude"])
    crs="EPSG:4326",
)
```

Data Cleaning

```
schools_df["School_Longitude"] = pd.to_numeric(schools_df["School_Longitude"], errors="coerce")
schools_df["School_Latitude"]  = pd.to_numeric(schools_df["School_Latitude"],  errors="coerce")
schools_df = schools_df.dropna(subset=["School_Longitude", "School_Latitude"]).copy()

schools_gdf = gpd.GeoDataFrame(
    schools_df,
    geometry=gpd.points_from_xy(schools_df["School_Longitude"], schools_df["School_Latitude"])
    crs="EPSG:4326",
)

schools_gdf["Total"] = (
    schools_gdf["Total"]
      .astype(str)
      .str.replace(",", "", regex=False)
)
schools_gdf["Total"] = pd.to_numeric(schools_gdf["Total"], errors="coerce")

schools_gdf[["Total"]].describe()
```

Threshold Definition

We define the threshold as the 0.75 quantile of Total enrollment.

```
q = 0.75
thr = schools_gdf["Total"].quantile(q)

schools_gdf["above_thr"] = schools_gdf["Total"] > thr
schools_gdf["above_thr"].value_counts(dropna=False)

schools_gdf["size_bin"] = pd.qcut(
    schools_gdf["Total"],
    q=4,
    duplicates="drop"
)
```

```
size_map = {b: s for b, s in zip(schools_gdf["size_bin"].cat.categories, [10, 25, 45, 70])}
schools_gdf["ms"] = schools_gdf["size_bin"].map(size_map)

schools_gdf[["Total", "size_bin", "ms"]].head()
```

Plot: Schools over Community Areas

```
COMM_FILL = "#dbeff2"
COMM_EDGE = "#111111"

fig, ax = plt.subplots(figsize=(10, 10))

# community polygons
comm_gdf.plot(
    ax=ax,
    facecolor="white",
    edgecolor="black",
    linewidth=1.6,
    zorder=1
)

# schools: below threshold (blue)
schools_gdf.loc[~schools_gdf["above_thr"]].plot(
    ax=ax,
    markersize=15,
    color="#2b6cb0",
    alpha=0.75,
    linewidth=0,
    label="Schools  threshold",
    zorder=2
)

# schools: above threshold (red)
schools_gdf.loc[schools_gdf["above_thr"]].plot(
    ax=ax,
    markersize=25,
    color="#d62828",
    alpha=0.85,
    linewidth=0,
    label="Schools > threshold",
    zorder=3
)

ax.set_title("Chicago Public Schools by Community Areas", fontsize=16, pad=14)
ax.set_axis_off()
ax.legend(
    title="Student Enrollment (75th Percentile Split)",
    labels=[
        "Enrollment  75th Percentile",
```

```
        "Enrollment > 75th Percentile"
    ],
    loc="lower left",
    frameon=True,
    fontsize=12,
    title_fontsize=13
)

plt.tight_layout()
plt.show()
```

Exporting High-Resolution PNG

For submission purposes, we export the figure as a high-resolution PNG file.

```
output_path = project_root / "data" / "derived-data" / "plot_app_spatial_scatter.png"

fig.savefig(output_path, dpi=300, bbox_inches='tight')
print(f"Saved plot as PNG: {output_path}")
```

3. Student Performance Gaps Among Different Structural Disadvantage Groups

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patheffects as path_effects
from pathlib import Path
import numpy as np

st.set_page_config(layout="wide")

st.title("IAR Proficiency Distribution")
st.write("")
st.write("### Comparison: ELA vs Math")
st.write("")

project_root = Path(__file__).resolve().parents[1]
raw_dir = project_root / "data" / "raw-data"

@st.cache_data
def load_data():
    df = pd.read_csv(raw_dir / "iar_proficiency.csv")
    df = df[df["School Code"] == "--"]
    df.columns = df.columns.str.strip()
    for col in ["%At or Above Proficient", "%Approaching Proficient", "%Below Proficient"]:
        df[col] = pd.to_numeric(df[col], errors="coerce")
    return df
```

```
df = load_data()

group_map = {
    "Economic Disadvantage": "Economic Disadvantage",
    "English Learner": "English Learner Status",
    "Gender": "Gender",
    "Race": "Race/Ethnicity",
}
group_choice = st.sidebar.selectbox("Select Subgroup (X Axis)", list(group_map.keys()))
group_col = group_map[group_choice]

def draw_proficiency_chart(data, title, subgroup_name):
    plot_df = data.copy()
    if subgroup_name == "Race":
        plot_df["Student Group"] = plot_df["Student Group"].replace({
            "White": "White",
            "Black/African American": "Black",
            "Latinx": "Latinx"
        })
        keep = ["White", "Black", "Latinx"]
        plot_df["Student Group"] = plot_df["Student Group"].apply(lambda x: x if x in keep else
        plot_df["Student Group"] = pd.Categorical(plot_df["Student Group"], categories=keep +

    plot_df = plot_df.groupby("Student Group").mean(numeric_only=True).reset_index()

    proficiency_order = ["%Below Proficient", "%Approaching Proficient", "%At or Above Profici
    colors = ["#e74c3c", "#f1c40f", "#2ecc71"]

    x_labels = plot_df["Student Group"]
    bottom = np.zeros(len(plot_df))

    fig, ax = plt.subplots(figsize=(8, 7))

    for i, prof in enumerate(proficiency_order):
        values = plot_df[prof].values
        bars = ax.bar(x_labels, values, bottom=bottom, color=colors[i], label=prof, edgecolor=

        for bar, val, btm in zip(bars, values, bottom):
            if val > 5:
                ax.text(
                    bar.get_x() + bar.get_width()/2,
                    btm + val/2,
                    f"{val:.1f}%",
                    ha="center", va="center", fontsize=9, color="black",
                    path_effects=[path_effects.withStroke(linewidth=2, foreground="white")]
                )
        bottom += values

    ax.set_ylabel("Percent")
    ax.set_title(title, fontsize=14, weight='bold', pad=15)
    ax.set_ylim(0, 105)
```

```
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.tick_params(axis='x', pad=15)
    ax.tick_params(axis='y', pad=15)

    handles, labels = ax.get_legend_handles_labels()
    leg = ax.legend(handles[::-1], labels[::-1], title="Proficiency",
                    loc='upper center', bbox_to_anchor=(0.5, -0.12),
                    ncol=3, fontsize=10, title_fontsize=10)
    leg._legend_box.sep = 9

    return fig

col1, col2 = st.columns(2)

ela_data = df[(df["Test Name"].str.contains("ELA", case=False, na=False)) & (df["Category"] ==
math_data = df[(df["Test Name"].str.contains("Math", case=False, na=False)) & (df["Category"] =

with col1:
    if not ela_data.empty:
        fig_ela = draw_proficiency_chart(ela_data, f"ELA Proficiency by {group_choice}", group_
        st.pyplot(fig_ela)
    else:
        st.warning("No ELA data found.")

with col2:
    if not math_data.empty:
        fig_math = draw_proficiency_chart(math_data, f"Math Proficiency by {group_choice}", gro
        st.pyplot(fig_math)
    else:
        st.warning("No Math data found.")
```

4. Relationship Between Resource Index and Opportunity Index

Regression Analysis: RI and OI

In this section, we examine the relationship between the **Resource Index (RI)** and the **Opportunity Index (OI)**.

Specifically, we estimate the following linear model:

RI = + OI +

This regression allows us to assess whether schools serving more disadvantaged populations (higher OI) tend to have systematically different resource levels (RI).

Environment Setup for Analysis

```
import pandas as pd
import altair as alt
from pathlib import Path
import statsmodels.api as sm
import tempfile
from IPython.display import Image, display
import vl_convert as vlc
import os

# Define project paths robustly for Quarto
project_root = Path(os.environ.get("QUARTO_PROJECT_DIR", Path.cwd())).resolve().parent
derived_dir = project_root / "data" / "derived-data"

# Load cleaned dataset
df = pd.read_csv(derived_dir / "cps_df.csv")
```

We load the previously constructed dataset containing RI and OI.

Data Cleaning for Regression

```
# Drop missing values in key variables
df_clean = df.dropna(subset=['OI', 'RI'])

len(df_clean)
```

We restrict the regression sample to schools with non-missing Resource Index and Opportunity Index values.

Ordinary Least Squares (OLS) Regression

```
# Define regression variables
X = sm.add_constant(df_clean['OI'])
y = df_clean['RI']

# Fit OLS model
model = sm.OLS(y, X).fit()

# Extract p-value for OI coefficient
p_value = model.pvalues['OI']
```

```
p_text = f"p-value: {p_value:.3e}"
slope = model.params[1]

model.summary()
```

This regression estimates the marginal association between OI and RI.
The p-value indicates whether the relationship is statistically significant.

Visualization: RI vs OI

To better illustrate the relationship, we construct a scatter plot with:

- Individual schools as points
- A fitted linear regression line
- The p-value displayed on the figure

```
# Scatter Plot

scatter = alt.Chart(df_clean).mark_point(
    filled=True,
    size=75,
    opacity=0.75,
    color="#1f77b4"
).encode(
    x=alt.X(
        'OI:Q',
        title='Opportunity Index (Structural Disadvantage)',
        axis=alt.Axis(labelFontSize=12, titleFontSize=14)
    ),
    y=alt.Y(
        'RI:Q',
        title='Resource Index (Hard + Soft School Resources)',
        axis=alt.Axis(labelFontSize=12, titleFontSize=14)
    ),
    tooltip=[
        alt.Tooltip('Short_Name:N', title='School'),
        alt.Tooltip('OI:Q', title='Opportunity Index', format=".2f"),
        alt.Tooltip('RI:Q', title='Resource Index', format=".2f")
    ]
)

# Regression Line

regression = scatter.transform_regression(
    'OI', 'RI'
).mark_line(
    color='black',
    size=2
```

```
)

# Annotation (Slope + p-value)

annotation_text = f"Slope = {slope:.2f} (p = {p_value:.3f})"

annotation = alt.Chart(
    pd.DataFrame({'text': [annotation_text]})
).mark_text(
    align='right',
    baseline='bottom',
    dx=-10,
    dy=-10,
    fontSize=11,
    color='gray'
).encode(
    x=alt.value(640),
    y=alt.value(440),
    text='text:N'
)

# Combine Layers

chart = (scatter + regression + annotation).properties(
    width=700,
    height=480,
    title={
        "text": "Do Higher-Need Schools Receive More Resources?",
        "subtitle": [
            "Opportunity Index measures structural disadvantage",
            "Resource Index combines facility utilization and 5Essentials survey scores"
        ],
        "fontSize": 18,
        "subtitleFontSize": 12
    }
)

chart
```

The scatter plot visually represents the direction and strength of the relationship between school disadvantage and resource allocation.

Exporting High-Resolution PNG

For submission purposes, we export the figure as a high-resolution PNG file.

```
# Convert chart to PNG bytes
png_bytes = vlc.vegalite_to_png(chart.to_dict(), scale=3)

# Save to derived-data directory
output_path = derived_dir / "plot_RI_vs_OI.png"
with open(output_path, "wb") as f:
    f.write(png_bytes)

print(f"Saved chart as PNG: {output_path}")
```

The image file can now be used in slides or included in the PDF submission.

Interpretation

If > 0: Schools serving more disadvantaged populations tend to have higher Resource Index values.

If < 0: Schools serving more disadvantaged populations tend to have lower Resource Index values.

Statistical significance (based on the p-value) determines whether this relationship is distinguishable from zero.

This analysis provides a first step toward evaluating whether resource allocation systematically aligns with student disadvantage.

5. Resource Index and Opportunity Index's Spatial Distribution, and Correlation with ELA and Math Score

```
import streamlit as st
import pandas as pd
import numpy as np
import geopandas as gpd
import matplotlib.pyplot as plt
import altair as alt

from pathlib import Path
from shapely import wkt
from shapely.geometry import Point
from scipy.stats import linregress

st.set_page_config(page_title="CPS Dashboard", layout="wide")

alt.data_transformers.disable_max_rows()

@st.cache_data
def load_data():
```

```
    try:
        project_root = Path(__file__).resolve().parent.parent
    except NameError:
        project_root = Path.cwd().parent

    raw_community = project_root / "data" / "raw-data" / "community_boundaries.csv"
    raw_schools = project_root / "data" / "derived-data" / "cps_df.csv"

    comm_df = pd.read_csv(raw_community, sep=";", encoding="utf-8-sig")
    schools_df = pd.read_csv(raw_schools)

    comm_df.columns = comm_df.columns.str.strip().str.strip('"')
    schools_df.columns = schools_df.columns.str.strip()

    if "the_geom" not in comm_df.columns:
        raise KeyError(f"'the_geom' not found. Columns found: {list(comm_df.columns)}")

    comm_df["geometry"] = comm_df["the_geom"].apply(wkt.loads)
    comm_gdf = gpd.GeoDataFrame(comm_df, geometry="geometry", crs="EPSG:4326")

    if "COMMUNITY" in comm_gdf.columns:
        comm_gdf["COMMUNITY"] = comm_gdf["COMMUNITY"].astype(str).str.strip()

    return schools_df, comm_gdf

def find_first_existing_column(df, candidates, label_for_error):
    for c in candidates:
        if c in df.columns:
            return c
    raise KeyError(
        f"Cannot find a valid column for {label_for_error}. "
        f"Tried: {candidates}. Available columns: {list(df.columns)}"
    )

def to_numeric_if_exists(df, cols):
    for c in cols:
        if c in df.columns:
            df[c] = pd.to_numeric(df[c], errors="coerce")
    return df

def prepare_school_geo(schools_df):
    df = schools_df.copy()
    lat_col = find_first_existing_column(df, ["School_Latitude", "Latitude", "lat", "LATITUDE"
    lon_col = find_first_existing_column(df, ["School_Longitude", "Longitude", "lon", "lng", "
    df = to_numeric_if_exists(df, [lat_col, lon_col])
    df = df.dropna(subset=[lat_col, lon_col]).copy()
    school_gdf = gpd.GeoDataFrame(
        df,
        geometry=[Point(xy) for xy in zip(df[lon_col], df[lat_col])],
        crs="EPSG:4326",
    )
```

```
    return school_gdf

def spatial_join_schools_to_communities(schools_df, comm_gdf):
    school_gdf = prepare_school_geo(schools_df)
    keep_cols = [c for c in ["COMMUNITY", "AREA_NUMBE", "AREA_NUM_1", "geometry"] if c in comm_
    comm_subset = comm_gdf[keep_cols].copy()
    joined = gpd.sjoin(school_gdf, comm_subset, how="left", predicate="within")
    return joined

def detect_metric_columns(cps_df):
    col_map = {}
    col_map["Resource Index"] = find_first_existing_column(
        cps_df, ["RI", "ri", "Readiness Index", "readiness_index"], "RI"
    )
    col_map["Opportunity Index"] = find_first_existing_column(
        cps_df, ["Opportunity Index", "OI", "oi", "opportunity_index"], "OI / Opportunity Index
    )
    col_map["ELA"] = find_first_existing_column(
        cps_df, ["IAR ELA %", "ELA %At or Above Proficient", "ELA % At or Above Proficient", "I
    )
    col_map["Math"] = find_first_existing_column(
        cps_df, ["IAR MATH %", "Math %At or Above Proficient", "MATH %At or Above Proficient",
    )
    return col_map

def get_school_name_column(df):
    for c in ["Short_Name", "Long_Name", "School_Name", "Name", "school_name", "SCHOOL_NM", "Sc
        if c in df.columns:
            return c
    return None

def compute_community_metric_mean(joined_gdf, comm_gdf, metric_col):
    tmp = joined_gdf.copy()
    tmp[metric_col] = pd.to_numeric(tmp[metric_col], errors="coerce")
    if "COMMUNITY" not in tmp.columns:
        raise KeyError("COMMUNITY column not found after spatial join.")
    mean_df = (
        tmp.dropna(subset=["COMMUNITY", metric_col])
        .groupby("COMMUNITY", as_index=False)[metric_col]
        .mean()
        .rename(columns={metric_col: "community_mean"})
    )
    merged = comm_gdf.copy()
    merged["COMMUNITY"] = merged["COMMUNITY"].astype(str).str.strip()
    merged = merged.merge(mean_df, on="COMMUNITY", how="left")
    return merged, mean_df

def plot_choropleth(comm_metric_gdf, metric_label):
    fig = plt.figure(figsize=(8, 8))
    gs = fig.add_gridspec(nrows=1, ncols=2, width_ratios=[0.06, 0.94], wspace=0.05)
    cax = fig.add_subplot(gs[0, 0])
```

```
    ax = fig.add_subplot(gs[0, 1])
    comm_metric_gdf.plot(
        column="community_mean",
        cmap="viridis",
        linewidth=0.7,
        edgecolor="black",
        legend=True,
        legend_kwds={"cax": cax},
        missing_kwds={"color": "lightgray", "edgecolor": "black", "hatch": "///", "label": "No
        ax=ax,
    )
    ax.set_title(f"{metric_label} Mean by Chicago Community", fontsize=14)
    ax.set_axis_off()
    cax.yaxis.set_ticks_position("left")
    cax.yaxis.set_label_position("left")
    plt.tight_layout()
    return fig

def build_scatter_df(df, x_col, y_col, x_label, y_label, school_name_col=None):
    plot_df = df.copy()
    needed_cols = [x_col, y_col]
    if school_name_col and school_name_col in plot_df.columns:
        needed_cols = [school_name_col] + needed_cols
    plot_df = plot_df[needed_cols].copy()
    plot_df[x_col] = pd.to_numeric(plot_df[x_col], errors="coerce")
    plot_df[y_col] = pd.to_numeric(plot_df[y_col], errors="coerce")
    plot_df = plot_df.dropna(subset=[x_col, y_col]).copy()
    if school_name_col and school_name_col in plot_df.columns:
        plot_df["School"] = plot_df[school_name_col].astype(str)
    else:
        plot_df["School"] = "Unknown"
    plot_df = plot_df.rename(columns={x_col: x_label, y_col: y_label})
    return plot_df[["School", x_label, y_label]].copy()

def compute_regression_stats(plot_df, x_label, y_label):
    if len(plot_df) < 3: return None
    x = pd.to_numeric(plot_df[x_label], errors="coerce")
    y = pd.to_numeric(plot_df[y_label], errors="coerce")
    valid = np.isfinite(x) & np.isfinite(y)
    x, y = x[valid], y[valid]
    if len(x) < 3: return None
    res = linregress(x, y)
    return {"slope": res.slope, "pvalue": res.pvalue, "n": int(len(x))}

def format_pvalue(p):
    if p is None or pd.isna(p): return "NA"
    return "< 0.001" if p < 0.001 else f"{p:.3f}"

def plot_scatter_altair(plot_df, x_label, y_label):
    points = alt.Chart(plot_df).mark_circle(size=60, opacity=0.65).encode(
        x=alt.X(f"{x_label}:Q", title=x_label),
```

```
        y=alt.Y(f"{y_label}:Q", title=y_label),
        tooltip=[alt.Tooltip("School:N"), alt.Tooltip(f"{x_label}:Q", format=".4f"), alt.Toolt
    )
    reg_line = points.transform_regression(x_label, y_label).mark_line(color="red", size=2)
    return (points + reg_line).properties(height=500, title=f"{y_label} vs {x_label}").interact

try:
    cps_df, comm_gdf = load_data()
    cps_df = to_numeric_if_exists(cps_df.copy(), ["RI", "OI", "Opportunity Index", "IAR ELA %"
    col_map = detect_metric_columns(cps_df)
    joined_schools_gdf = spatial_join_schools_to_communities(cps_df, comm_gdf)
except Exception as e:
    st.error(f"Data preparation failed: {e}")
    st.stop()

st.sidebar.header("Navigation")
main_section = st.sidebar.selectbox("Go to", ["Introduction", "Spatial Distribution", "Scatter

if main_section == "Introduction":
    st.title("CPS Dashboard: Introduction")
    st.markdown(
    """
    ##   Indices Construction

    ### **Resource Index (RI)**

    **Hard Resources**

    Let:

    - $U_i$ = space utilization rate of school $i$
    - $c = 0.85$ = efficiency center

    Nonlinear crowding penalty centered at optimal utilization:

    $$
    Hard_i = - (U_i - c)^2
    $$

    Standardized:

    $$
    Hard_i^{*} = \\frac{Hard_i - \\mu_{Hard}}{\\sigma_{Hard}}
    $$

    **Soft Resources** - derived from five 5Essentials dimensions:

    - Ambitious Instruction (AI)
    - Effective Leaders (EL)
    - Collaborative Teachers (CT)
    - Involved Families (IF)
```

```
    - Supportive Environment (SE)

    Each component standardized (z-score) and averaged:

    $$
    Soft_i = \\frac{Z_{AI,i} + Z_{EL,i} + Z_{CT,i} + Z_{IF,i} + Z_{SE,i}}{5}
    $$

    **Final Resource Index**:

    $$
    RI_i = 0.5 \\cdot Hard_i^{*} + 0.5 \\cdot Soft_i
    $$

    ---

    ### **Opportunity Index (OI)**

    Captures cumulative disadvantage exposure:

    - % Economically Disadvantaged (ED)
    - % English Learners (EL)
    - % Students with Disabilities (DIS)
    - Simpson Diversity Concentration Index (RACE)

    Standardized and averaged:

    $$
    OI = \\frac{Z_{ED} + Z_{ELL} + Z_{DIS} + Z_{RACE}}{4}
    $$

    > Higher OI indicates greater structural disadvantage.
    """,
        unsafe_allow_html=True
    )

elif main_section == "Spatial Distribution":
    st.header("Spatial Distribution")
    spatial_metric = st.selectbox("Select indicator", ["Resource Index", "Opportunity Index"],
    metric_col = col_map[spatial_metric]

    left, right = st.columns([1.2, 1])

    try:
        comm_metric_gdf, comm_mean_df = compute_community_metric_mean(joined_schools_gdf, comm_
        fig_map = plot_choropleth(comm_metric_gdf, spatial_metric)
        display_mean_df = comm_mean_df.rename(columns={"community_mean": f"{spatial_metric} Ave

        if spatial_metric == "Resource Index":
            display_mean_df = display_mean_df.sort_values(by=f"{spatial_metric} Average", ascen
            table_title = f"**Bottom 10 communities (lowest {spatial_metric})**"
```

```
        else:
            display_mean_df = display_mean_df.sort_values(by=f"{spatial_metric} Average", ascen
            table_title = f"**Top 10 communities (highest {spatial_metric})**"

        display_mean_df = display_mean_df.reset_index(drop=True)
        display_mean_df.index = range(1, len(display_mean_df) + 1)

        with left:
            st.markdown(f"<h2 style='text-align: center;'>{spatial_metric} Map</h2>", unsafe_al
            st.pyplot(fig_map, use_container_width=True)

        with right:
            st.markdown(f"<h2 style='text-align: center;'>Community Averages</h2>", unsafe_allo
            st.markdown(table_title)
            st.dataframe(display_mean_df.head(10), use_container_width=True)
    except Exception as e:
        st.error(f"Failed to build spatial section: {e}")

elif main_section == "Scatter Plot":
    st.header("Scatter Plot Analysis")
    c1, c2 = st.columns(2)
    with c1:
        x_choice = st.selectbox("Select index", ["Resource Index", "Opportunity Index"], key="s
    with c2:
        y_choice = st.selectbox("Select score", ["ELA", "Math"], key="scatter_y")

    x_col, y_col = col_map[x_choice], col_map[y_choice]
    name_col = get_school_name_column(cps_df)

    try:
        scatter_plot_df = build_scatter_df(cps_df, x_col, y_col, x_choice, y_choice, name_col)
        alt_chart = plot_scatter_altair(scatter_plot_df, x_choice, y_choice)
        reg_stats = compute_regression_stats(scatter_plot_df, x_choice, y_choice)

        st.markdown(f"<h2 style='text-align: center;'>{y_choice} vs {x_choice}</h2>", unsafe_al
        st.altair_chart(alt_chart, use_container_width=True)

        if reg_stats:
            p_val, slope = format_pvalue(reg_stats["pvalue"]), reg_stats["slope"]
            interp = "Low correlation suggests potential resource misalignment." if abs(slope)
            st.markdown(f"""
                <div style="font-size: 0.9rem; color: #666; text-align: left;">
                    Linear regression: slope = {slope:.3f}, p-value = {p_val}, n = {reg_stats[
                    {interp}
                </div>
            """, unsafe_allow_html=True)
    except Exception as e:
        st.error(f"Failed to build scatter plot: {e}")
```